# Data-Mart-Erstellung in SQL

## IMPLEMENTIERUNG EINER RELATIONALEN DATENBANK FÜR EINE BUCHTAUSCH-APP

Portfolio DLBDSPBDM01_D Phase 2
Tutorin: Anna Androvitsanea
Felix Stellbrink IU14127320
B.Sc. Informatik, 11.11.2026

# Gliederung


Entitäten:

- ort_____3
- nutzer_____4
- sprache_____5
- genre_____6
- autor_____7
- buch_____8
- buch_genre_____9
- buch_autor_____10
- uebergabeart_____11
- exemplar_____12
- anfrage_____13
- bewertung_____14
- ausleihe_____15

Trigger:

- trg_bewertung_____16
- trg_verfuegbarkeit_____17
- trg_status_____18
- trg_anfrage_____19

Indizes:

- ausleihe_aktiv_____20
- anfrage_angenommen____20

Abschluss:

- Datenbestand_____21
- Zusammenfassung_____26

JOINs:

- Aktive Ausleihen pro Nutzer ___22
- Überfällige Ausleihen_____23
- Durchschn. Bewertungen_____24
- Verfügbare Bücher_____25

# Entität: ort

Referenzdaten · 10 Einträge

## Definition

```
CREATE TABLE ort (
    postleitzahl CHAR(5) PRIMARY KEY,
    stadt VARCHAR(100) NOT NULL,
    bundesland VARCHAR(50) NOT NULL
);
```

## Erläuterung

Als Primärschlüssel dient die Postleitzahl. Sie ist eindeutiger als die Stadt, denn große Städte wie Hamburg, Berlin und München haben mehrere Postleitzahlen.
Der Ort hätte auch als Spalte in der Tabelle nutzer geführt werden können. Da dort jedoch viele Duplikate entstanden wären, wurde er als eigene Tabelle umgesetzt.
Als Datentyp wurde CHAR statt INT gewählt, weil Postleitzahlen mit einer Null beginnen können. INT würde diese Null entfernen. Aus 01067 würde 1067 werden.

## Negativer Testfall T1 · PRIMARY KEY · erwartet 23505

```
1  INSERT INTO ort (postleitzahl,stadt,bundesland)
2  VALUES
3  ('22767', 'Hamburg','Hamburg')
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "ort_pkey"
Key (postleitzahl)=(22767) already exists.

SQL state: 23505
Detail: Key (postleitzahl)=(22767) already exists.
```

## Positiver Testfall P1 · CHAR(5) erhält die führende Null

```
SELECT postleitzahl,
       CAST(postleitzahl AS INT) AS als_zahl
FROM ort WHERE postleitzahl = '01067';
```

| | postleitzahl [PK] character (5) | als_zahl integer |
|---|---|---|
| 1 | 01067 | 1067 |

# Entität: nutzer

Personendaten · 10 Einträge

## Definition

```
CREATE TABLE nutzer (
    nutzer_id SERIAL PRIMARY KEY,
    vorname VARCHAR(50) NOT NULL,
    nachname VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE CHECK
    (email LIKE '%@%') NOT NULL,
    passwort_hash VARCHAR(255) NOT NULL,
    strasse VARCHAR(100),
    hausnummer VARCHAR(10),
    postleitzahl CHAR(5) NOT NULL,
    FOREIGN KEY (postleitzahl)
    REFERENCES ort (postleitzahl)
);
```

## Erläuterung

Als Primärschlüssel dient der künstliche Schlüssel nutzer_id. SERIAL vergibt die Nummern automatisch.
Der Ort wird hier durch die PLZ referenziert, Straße und Hausnummer sind optional. Die genaue Adresse braucht man erst beim Versand, die Postleitzahl jedoch immer für die Filterung.
CHECK bei email sorgt dafür, dass nur Adressen mit „@“ akzeptiert werden. UNIQUE sorgt dafür, dass die E-Mail eindeutig ist.

## Negativer Testfall T2 · CHECK email · erwartet 23514

```
1  INSERT INTO nutzer (vorname,nachname,email,passwort_hash,strasse,hausnummer,postleitzahl)
2  VALUES
3  ('Peter','Maffai','P.Maffai.com','hashwert','Musterstrasse','1a','01067')
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "nutzer" violates check constraint "nutzer_email_check"
Failing row contains (11, Peter, Maffai, P.Maffai.com, hashwert, Musterstrasse, 1a, 01067).

SQL state: 23514
Detail: Failing row contains (11, Peter, Maffai, P.Maffai.com, hashwert, Musterstrasse, 1a, 01067).
```

## Positiver Testfall P2 · Adresse ist optional, PLZ ist Pflicht

```
SELECT hausnummer, strasse, postleitzahl
    FROM nutzer WHERE strasse IS NULL;
```

| | hausnummer character varying (10) | strasse character varying (100) | postleitzahl character (5) |
|---|---|---|---|
| 1 | [null] | [null] | 01067 |
| 2 | [null] | [null] | 20457 |

# Entität: sprache

Referenzdaten · 12 Einträge

## Definition

```
CREATE TABLE sprache (
    sprach_id SERIAL PRIMARY KEY,
    sprache VARCHAR(20) UNIQUE NOT NULL
);
```

## Erläuterung

Sprache als eigene Tabelle wurde angelegt, damit nachträglich mit einem INSERT Sprachen hinzugefügt werden können. Als CHECK in der Tabelle buch hätte jede neue Sprache eine Schemaänderung per ALTER TABLE erfordert.
Als Primärschlüssel dient der künstliche Schlüssel sprach_id.
SERIAL vergibt die Nummern automatisch.
UNIQUE verhindert, dass dieselbe Sprache zweimal angelegt wird.
Die Sprache „Sonstiges“ dient als Auffangwert für Bücher, deren Sprache nicht in der Liste enthalten ist.

## Negativer Testfall T3 · UNIQUE · erwartet 23505

```
1  INSERT INTO sprache (sprache)
2  VALUES
3  ('Deutsch');
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "sprache_sprache_key"
Key (sprache)=(Deutsch) already exists.

SQL state: 23505
Detail: Key (sprache)=(Deutsch) already exists.
```

## Positiver Testfall P3 · Auffangwert für Sprachen außerhalb der Liste

```
SELECT * FROM sprache
    WHERE sprache ='Sonstiges';
```

| | sprach_id [PK] integer | sprache character varying (20) |
|---|---|---|
| 1 | 12 | Sonstiges |

# Entität: genre

Referenzdaten · 13 Einträge

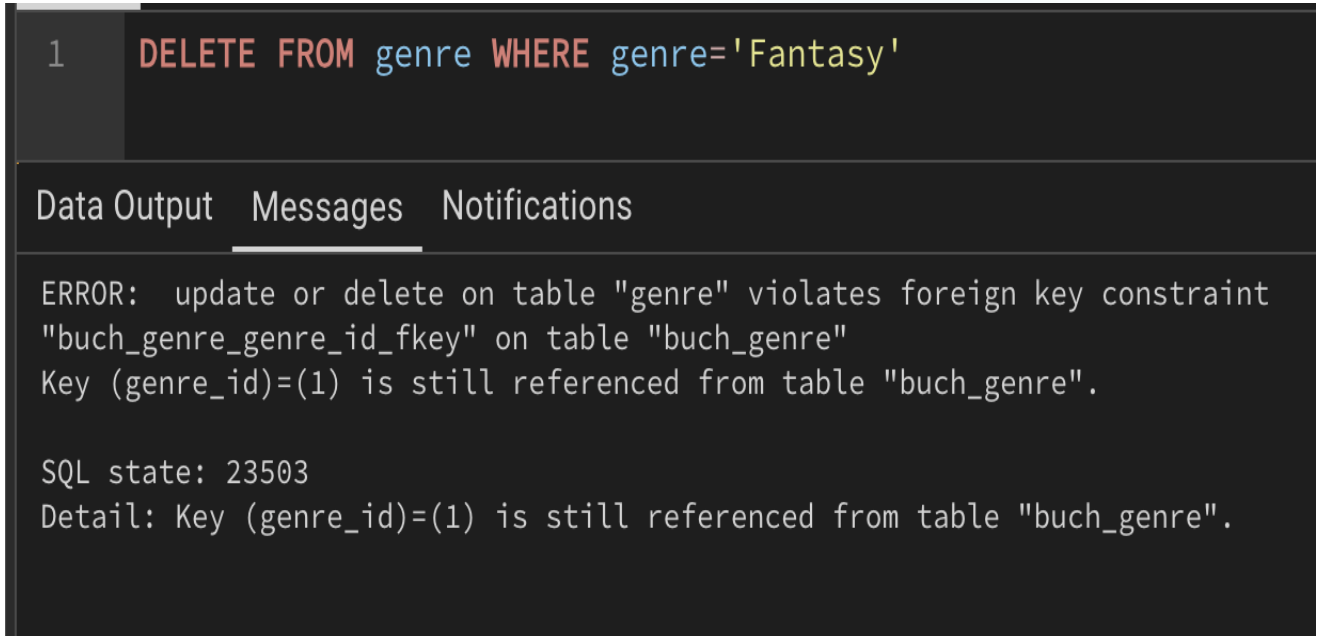
## Definition

```
CREATE TABLE genre (
    genre_id SERIAL PRIMARY KEY,
    genre VARCHAR(50) UNIQUE NOT NULL
);
```

## Erläuterung

Genre als eigene Tabelle wurde angelegt, damit nachträglich mit einem INSERT Genres hinzugefügt werden können. Als CHECK in der Tabelle buch hätte jedes neue Genre eine Schemaänderung per ALTER TABLE erfordert.
Als Primärschlüssel dient der künstliche Schlüssel genre_id.
SERIAL vergibt die Nummern automatisch.
UNIQUE verhindert, dass dasselbe Genre zweimal angelegt wird.
Das Genre „Sonstige“ dient als Auffangwert für Bücher, deren Genre nicht in der Liste enthalten ist.

**Negativer Testfall** T4 · FOREIGN KEY aus buch_genre · erwartet 23503



# Entität: autor

Bibliografische Daten · 12 Einträge

## Definition

```
CREATE TABLE autor (
    autor_id SERIAL PRIMARY KEY,
    vorname VARCHAR(50) NOT NULL,
    nachname VARCHAR(50) NOT NULL
);
```

## Erläuterung

Autor wurde als eigene Tabelle angelegt, da ein Autor mehrere Bücher schreiben kann.
Als Primärschlüssel dient der künstliche Schlüssel autor_id.
SERIAL vergibt die Nummern automatisch.
Auf Vorname und Nachname ist hier kein UNIQUE, da mehrere Autoren denselben Vor- oder Nachnamen haben können.

**Negativer Testfall** T5 · FOREIGN KEY aus buch_autor · erwartet 23503

```
1   DELETE FROM autor WHERE nachname='Fitzek'
```

Data Output | Messages | Notifications

```
ERROR:  update or delete on table "autor" violates foreign key constraint
"buch_autor_autor_id_fkey" on table "buch_autor"
Key (autor_id)=(4) is still referenced from table "buch_autor".

SQL state: 23503
Detail: Key (autor_id)=(4) is still referenced from table "buch_autor".
```

# Entität: buch

Bibliografische Daten · 12 Einträge

## Definition

```
CREATE TABLE buch (
    isbn CHAR(13) PRIMARY KEY,
    titel VARCHAR(200) NOT NULL,
    erscheinungsjahr SMALLINT NOT NULL
    CHECK (erscheinungsjahr
           BETWEEN 1440 AND 2100),
    sprach_id INT NOT NULL,
    FOREIGN KEY (sprach_id)
        REFERENCES sprache (sprach_id)
);
```

**Negativer Testfall** T6 · FOREIGN KEY sprach_id · erwartet 23503

```
1  INSERT INTO buch (isbn, titel, erscheinungsjahr, sprach_id)
2  VALUES
3  ('9876543210123','Testfälle','2023',76)
```

Data Output | Messages | Notifications

```
ERROR:  insert or update on table "buch" violates foreign key constraint
"buch_sprach_id_fkey"
Key (sprach_id)=(76) is not present in table "sprache".

SQL state: 23503
Detail: Key (sprach_id)=(76) is not present in table "sprache".
```

## Erläuterung

Als Primärschlüssel dient die ISBN, da sie für jedes Buch eindeutig ist.
CHAR(13) wurde verwendet, damit führende Nullen erhalten bleiben.
Das Erscheinungsjahr wurde bewusst auf 1440 bis 2100 begrenzt, da der erste Buchdruck 1440 startete und auch sehr alte Bücher erfasst werden können. 2100 hat den Vorteil, dass nicht jedes Jahr der CHECK der Tabelle angepasst werden muss.
sprach_id ist der Fremdschlüssel zur Tabelle sprache.

# Entität: buch_genre

Zwischentabelle N:M · 18 Einträge

## Definition

```
CREATE TABLE buch_genre (
    isbn CHAR(13) NOT NULL,
    genre_id INT NOT NULL,
    PRIMARY KEY (isbn, genre_id),
    FOREIGN KEY (isbn)
    REFERENCES buch(isbn)
        ON DELETE CASCADE,
    FOREIGN KEY (genre_id)
    REFERENCES genre (genre_id)
);
```

## Erläuterung

Diese Tabelle führt buch und genre zusammen, da ein Buch mehrere Genres haben kann und ein Genre mehreren Büchern zugeordnet sein kann.
Der Primärschlüssel ist aus den beiden Fremdschlüsseln isbn und genre_id zusammengesetzt.
ON DELETE CASCADE löscht mit einem Buch auch seine Genre-Zuordnungen. Ein Genre lässt sich dagegen nicht löschen, solange ihm noch Bücher zugeordnet sind.

## Negativer Testfall T7 · Zusammengesetzter PK · erwartet 23505

```
1  INSERT INTO buch_genre (isbn, genre_id)
2  VALUES
3  ('9783453530911',2)
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "buch_genre_pkey"
Key (isbn, genre_id)=(9783453530911, 2) already exists.

SQL state: 23505
Detail: Key (isbn, genre_id)=(9783453530911, 2) already exists.
```

## Positiver Testfall P4 · ein Buch mit verschiedenen Genres

```
SELECT * FROM buch_genre
    WHERE isbn='9783453530911';
```

| | isbn [PK] character (13) | genre_id [PK] integer |
|---|---|---|
| 1 | 9783453530911 | 1 |
| 2 | 9783453530911 | 2 |

# Entität: buch_autor

Zwischentabelle N:M · 13 Einträge

## Definition

```
CREATE TABLE buch_autor (
    isbn CHAR(13) NOT NULL,
    autor_id INT NOT NULL,
    PRIMARY KEY (isbn, autor_id),
    FOREIGN KEY (isbn)
    REFERENCES buch(isbn)
        ON DELETE CASCADE,
    FOREIGN KEY (autor_id)
    REFERENCES autor (autor_id)
);
```

## Erläuterung

Diese Tabelle führt buch und autor zusammen, da ein Buch mehrere Autoren haben kann und ein Autor mehreren Büchern zugeordnet sein kann.
Der Primärschlüssel ist aus den beiden Fremdschlüsseln isbn und autor_id zusammengesetzt.
ON DELETE CASCADE löscht mit einem Buch auch seine Autor-Zuordnungen. Ein Autor lässt sich dagegen nicht löschen, solange ihm noch Bücher zugeordnet sind.

## Negativer Testfall T8 · Zusammengesetzter PK · erwartet 23505

```
1  INSERT INTO buch_autor (isbn, autor_id)
2  VALUES
3  ('9780765326355',1)
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "buch_autor_pkey"
Key (isbn, autor_id)=(9780765326355, 1) already exists.

SQL state: 23505
Detail: Key (isbn, autor_id)=(9780765326355, 1) already exists.
```

## Positiver Testfall P5 · ein Autor mit zwei Büchern

```
SELECT * FROM buch_autor
    WHERE autor_id=1;
```

| | isbn [PK] character (13) | autor_id [PK] integer |
|---|---|---|
| 1 | 9780765326355 | 1 |
| 2 | 9783492705042 | 1 |

# Entität: uebergabeart

Referenzdaten · 2 Einträge

## Definition

```
CREATE TABLE uebergabeart(
    art_id SERIAL PRIMARY KEY,
    uebergabe VARCHAR(20) NOT NULL UNIQUE
    CHECK(uebergabe IN
    ('Abholung','Versand'))
);
```

## Erläuterung

Als Primärschlüssel ist hier der künstliche Schlüssel art_id definiert.
Der CHECK verhindert, dass andere Arten angegeben werden.
Bücher können nur abgeholt oder versandt werden.
Diese Tabelle enthält nur zwei Einträge, da die Wertemenge mit Abholung und Versand abgeschlossen ist.

**Negativer Testfall** T9 · CHECK · erwartet 23514

```
1  INSERT INTO uebergabeart (uebergabe)
2  VALUES
3  ('Flugzeug')
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "uebergabeart" violates check constraint
"uebergabeart_uebergabe_check"
Failing row contains (3, Flugzeug).

SQL state: 23514
Detail: Failing row contains (3, Flugzeug).
```

**Übersicht uebergabeart**

```
SELECT * FROM uebergabeart;
```

| | art_id [PK] integer | uebergabe character varying (20) |
|---|---|---|
| 1 | 1 | Versand |
| 2 | 2 | Abholung |

# Entität: exemplar

Bestandsdaten · 10 Einträge

## Definition

```
CREATE TABLE exemplar (
    exemplar_id SERIAL PRIMARY KEY,
    status VARCHAR(20) NOT NULL
    CHECK(status IN ('Verfuegbar', 'Verliehen',
        'Archiviert')),
    zustand VARCHAR(20) NOT NULL
      CHECK(zustand IN ('Hervorragend',
            'Gut','Schlecht')),
    nutzer_id INT NOT NULL,
    isbn CHAR(13) NOT NULL,
    FOREIGN KEY (nutzer_id) REFERENCES nutzer(nutzer_id),
    FOREIGN KEY (isbn) REFERENCES buch(isbn)
);
```

## Erläuterung

buch beschreibt das Werk, exemplar das einzelne physische Objekt. Dasselbe Buch kann bei mehreren Nutzern liegen, mit jeweils eigenem Zustand und Status.
Primärschlüssel ist der künstliche Schlüssel exemplar_id.
CHECK verhindert, dass andere Status- und Zustandswerte angegeben werden.

## Negativer Testfall T10 · CHECK status · erwartet 23514

```
1  INSERT INTO exemplar(status, zustand, nutzer_id, isbn)
2  VALUES
3  ('Kaputt', 'Schlecht',10,9780765326355)
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "exemplar" violates check constraint
"exemplar_status_check"
Failing row contains (20, Kaputt, Schlecht, 10, 9780765326355).

SQL state: 23514
Detail: Failing row contains (20, Kaputt, Schlecht, 10, 9780765326355).
```

## Positiver Testfall P6 · dasselbe Buch mit zwei Nutzern

```
SELECT exemplar_id, status, zustand, nutzer_id
    FROM exemplar WHERE isbn = '9780765326355';
```

| | exemplar_id [PK] integer | status character varying (20) | zustand character varying (20) | nutzer_id integer |
|---|---|---|---|---|
| 1 | 10 | Verfuegbar | Gut | 4 |
| 2 | 9 | Verliehen | Hervorragend | 1 |

# Entität: anfrage

Vorgangsdaten · 11 Einträge

## Definition

```
CREATE TABLE anfrage (
    anfrage_id SERIAL PRIMARY KEY,
    status VARCHAR(20) NOT NULL
    CHECK (status IN ('Gestellt',
    'Angenommen','Abgelehnt')),
    nutzer_id INT NOT NULL,
    exemplar_id INT NOT NULL,
    art_id INT NOT NULL,
    FOREIGN KEY(nutzer_id) REFERENCES nutzer(nutzer_id),
    FOREIGN KEY(exemplar_id) REFERENCES
        exemplar(exemplar_id),
    FOREIGN KEY(art_id) REFERENCES uebergabeart(art_id)
);
```

## Erläuterung

Mehrere Nutzer können dasselbe Exemplar anfragen. Der Besitzer nimmt eine Anfrage an und lehnt die anderen ab. Da eine Anfrage auch abgelehnt werden kann, ist sie eine eigene Entität und nicht Teil der Ausleihe.
Primärschlüssel ist der künstliche Schlüssel anfrage_id.
CHECK verhindert, dass ein anderer Status angegeben wird.

## Negativer Testfall T11 · CHECK status · erwartet 23514

```
1  INSERT INTO anfrage (status, nutzer_id, exemplar_id,art_id)
2  VALUES
3  ('Vielleicht',1,10,1)
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "anfrage" violates check constraint
"anfrage_status_check"
Failing row contains (12, Vielleicht, 1, 10, 1).

SQL state: 23514
Detail: Failing row contains (12, Vielleicht, 1, 10, 1).
```

## Positiver Testfall P7 · drei Anfragen auf dasselbe Exemplar

```
SELECT * FROM anfrage
    WHERE exemplar_id=13;
```

| | anfrage_id [PK] integer | status character varying (20) | nutzer_id integer | exemplar_id integer | art_id integer |
|---|---|---|---|---|---|
| 1 | 3 | Angenommen | 2 | 13 | 1 |
| 2 | 8 | Gestellt | 10 | 13 | 1 |
| 3 | 10 | Abgelehnt | 4 | 13 | 2 |

# Entität: bewertung

Vorgangsdaten · 10 Einträge

## Definition

```
CREATE TABLE bewertung(
    bewertung_id SERIAL PRIMARY KEY,
    sterne SMALLINT NOT NULL
    CHECK (sterne BETWEEN 1 AND 5),
    kommentar VARCHAR(500),
    ausleih_id INT UNIQUE NOT NULL,
    FOREIGN KEY(ausleih_id)
    REFERENCES ausleihe(ausleih_id)
);
```

## Erläuterung

Als Primärschlüssel ist hier der künstliche Schlüssel bewertung_id definiert.
CHECK stellt sicher, dass die Bewertung nur zwischen einem und fünf Sternen erfolgt.
UNIQUE bei ausleih_id stellt sicher, dass nur eine Bewertung pro Ausleihe erfolgen kann.
Kommentare sind optional. Bewertung verweist auf Ausleihe, nicht auf den Nutzer. Dadurch kann nur bewerten, wer das Exemplar tatsächlich ausgeliehen hat.

## Negativer Testfall T12 · 6 Sterne · erwartet 23514

```
1  INSERT INTO bewertung (sterne, kommentar, ausleih_id)
2  VALUES
3  (6,NULL,10)
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "bewertung" violates check constraint
"bewertung_sterne_check"
Failing row contains (13, 6, null, 10).

SQL state: 23514
Detail: Failing row contains (13, 6, null, 10).
```

## Positiver Testfall P8 · Kommentar optional

```
SELECT * FROM bewertung
WHERE kommentar IS NULL;
```

| | bewertung_id [PK] integer | sterne smallint | kommentar character varying (500) | ausleih_id integer |
|---|---|---|---|---|
| 1 | 3 | 5 | [null] | 3 |
| 2 | 6 | 4 | [null] | 6 |
| 3 | 7 | 5 | [null] | 7 |
| 4 | 10 | 5 | [null] | 10 |

# Entität: ausleihe

Vorgangsdaten · 13 Einträge

## Definition

```
CREATE TABLE ausleihe(
    ausleih_id SERIAL PRIMARY KEY,
    startdatum DATE NOT NULL,
    enddatum DATE,
    ausleihdauer INT NOT NULL CHECK (ausleihdauer >0),
    nutzer_id INT NOT NULL,
    exemplar_id INT NOT NULL,
    art_id INT NOT NULL,
    FOREIGN KEY(nutzer_id) REFERENCES nutzer(nutzer_id),
    FOREIGN KEY(exemplar_id) REFERENCES
        exemplar(exemplar_id),
    FOREIGN KEY(art_id) REFERENCES uebergabeart(art_id),
         CHECK(startdatum <= enddatum)
);
```

## Erläuterung

Als Primärschlüssel ist hier der künstliche Schlüssel ausleih_id definiert. CHECK stellt sicher, dass das Enddatum nicht vor dem Startdatum liegt. Enddatum ist optional. ausleihdauer beschreibt die vereinbarte Dauer, das enddatum hingegen das tatsächliche Ende der Ausleihe.

## Negativer Testfall T13 · enddatum < startdatum · erwartet 23514

```
1  INSERT INTO ausleihe (startdatum, enddatum, ausleihdauer, nutzer_id,
2                        exemplar_id, art_id)
3  VALUES
4  ('2026-08-12','2026-08-08',14,3,3,1)
```

Data Output | Messages | Notifications

```
ERROR:  new row for relation "ausleihe" violates check constraint "ausleihe_check"
Failing row contains (14, 2026-08-12, 2026-08-08, 14, 3, 3, 1).

SQL state: 23514
Detail: Failing row contains (14, 2026-08-12, 2026-08-08, 14, 3, 3, 1).
```

## Positiver Testfall P9 · Enddatum NULL

```
SELECT * FROM ausleihe
    WHERE enddatum IS NULL;
```

| | ausleih_id [PK] integer | startdatum date | enddatum date | ausleihdauer integer | nutzer_id integer | exemplar_id integer | art_id integer |
|---|---|---|---|---|---|---|---|
| 1 | 11 | 2026-09-20 | [null] | 21 | 5 | 9 | 1 |
| 2 | 12 | 2026-09-28 | [null] | 14 | 9 | 15 | 1 |
| 3 | 13 | 2026-10-01 | [null] | 28 | 3 | 16 | 1 |

# Trigger: trg_bewertung

## Definition

```
CREATE OR REPLACE FUNCTION pruefe_bewertung()
RETURNS TRIGGER AS $$
BEGIN
IF (SELECT enddatum FROM ausleihe
     WHERE ausleih_id =NEW.ausleih_id) IS NULL
THEN RAISE EXCEPTION 'Ausleihe % ist noch nicht
beendet, Bewertung nicht moeglich',
NEW.ausleih_id;
END IF; RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bewertung
BEFORE INSERT OR UPDATE ON bewertung
FOR EACH ROW
EXECUTE FUNCTION pruefe_bewertung();
```

## Erläuterung

Der Trigger lässt eine Bewertung erst zu, wenn die Ausleihe abgeschlossen ist. Ein CHECK reicht dafür nicht. Er sieht nur die Zeile, die gerade eingefügt oder geändert wird. Das Enddatum steht aber in der Tabelle ausleihe. Dorthin kann nur der Trigger schauen, weil er seine eigene Abfrage ausführt. Ohne UPDATE könnten bereits vergebene Bewertungen nachträglich einer noch laufenden Ausleihe zugeordnet werden.

**Negativer Testfall** T14 · INSERT · erwartet P0001

```
1  INSERT INTO bewertung (sterne, kommentar, ausleih_id)
2  VALUES
3  (4,NULL,12)
```

Data Output | Messages | Notifications

```
ERROR:  Ausleihe 12 ist noch nicht beendet, Bewertung nicht moeglich
CONTEXT:  PL/pgSQL function pruefe_bewertung() line 5 at RAISE

SQL state: P0001
```

**Negativer Testfall** T15 · UPDATE · erwartet P0001

```
1  UPDATE bewertung SET ausleih_id=11
2  WHERE bewertung_id=1;
```

Data Output | Messages | Notifications

```
ERROR:  Ausleihe 11 ist noch nicht beendet, Bewertung nicht moeglich
CONTEXT:  PL/pgSQL function pruefe_bewertung() line 5 at RAISE

SQL state: P0001
```

# Trigger: trg_verfuegbarkeit

## Definition

```
CREATE OR REPLACE FUNCTION
pruefe_verfuegbarkeit()
RETURNS TRIGGER AS $$
BEGIN
IF (SELECT status FROM exemplar
    WHERE exemplar_id=NEW.exemplar_id)
    <> 'Verfuegbar'
THEN RAISE EXCEPTION 'Exemplar % ist nicht
verfuegbar.',NEW.exemplar_id;
END IF; RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_verfuegbarkeit
BEFORE INSERT ON ausleihe
FOR EACH ROW
EXECUTE FUNCTION pruefe_verfuegbarkeit();
```

**Negativer Testfall** T16 · TRIGGER trg_verfuegbarkeit · erwartet P0001

```
1  INSERT INTO ausleihe (startdatum, enddatum, ausleihdauer, nutzer_id, exemplar_id, art_id)
2  VALUES ('2026-09-28', NULL, 19, 2, 9, 2);
```

Data Output | Messages | Notifications

```
ERROR:  Exemplar 9 ist nicht verfuegbar.
CONTEXT:  PL/pgSQL function pruefe_verfuegbarkeit() line 5 at RAISE

SQL state: P0001
```

## Erläuterung

Der Trigger lässt eine Ausleihe erst zu, wenn das Exemplar als Verfuegbar markiert ist. Ein CHECK reicht dafür nicht. Er sieht nur die Zeile, die gerade eingefügt wird. Der Status steht aber in der Tabelle exemplar. Dorthin kann nur der Trigger schauen, weil er seine eigene Abfrage ausführt.

# Trigger: trg_status

## Definition

```
CREATE OR REPLACE FUNCTION setze_status()
RETURNS TRIGGER AS $$
BEGIN
IF NEW.enddatum IS NULL
    THEN UPDATE exemplar SET
    status='Verliehen'
    WHERE exemplar_id=NEW.exemplar_id;

ELSE
    UPDATE exemplar SET status='Verfuegbar'
    WHERE exemplar_id=NEW.exemplar_id
    AND status='Verliehen';
END IF; RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_status
AFTER INSERT OR UPDATE ON ausleihe
FOR EACH ROW
EXECUTE FUNCTION setze_status();
```

## Positiver Testfall P10 · Status angepasst

```
UPDATE ausleihe SET enddatum = '2026-10-16' WHERE
ausleih_id = 11;
SELECT exemplar_id, status FROM exemplar WHERE
exemplar_id = 9;
```

| | exemplar_id [PK] integer | status character varying (20) |
|---|---|---|
| 1 | 9 | Verfuegbar |

## Erläuterung

Wenn kein Enddatum existiert, setzt der Trigger den Status in exemplar auf Verliehen. Wenn ein Enddatum existiert, wird geprüft, ob das Exemplar Verliehen ist. Damit wird verhindert, dass archivierte Exemplare wieder als Verfuegbar angezeigt werden. Beim Ausführen des Installationsskripts werden die Exemplare 9, 15 und 16 auf Verliehen gesetzt. Das Skript selbst enthält kein manuelles UPDATE mehr.
Der Status liegt in exemplar, der Auslöser in ausleihe. Deshalb ist ein Trigger nötig. AFTER statt BEFORE, da trg_status als BEFORE-Trigger alphabetisch vor trg_verfuegbarkeit liefe und den Status setzen würde, bevor überhaupt geprüft ist, ob das Exemplar frei ist.

# Trigger: trg_anfrage

## Definition

```
CREATE OR REPLACE FUNCTION pruefe_anfrage()
RETURNS TRIGGER AS $$
BEGIN
IF
(SELECT nutzer_id FROM exemplar WHERE
exemplar_id = NEW.exemplar_id)
= NEW.nutzer_id
THEN RAISE EXCEPTION 'Nutzer % besitzt dieses
Exemplar.', NEW.nutzer_id;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_anfrage
BEFORE INSERT OR UPDATE ON anfrage
FOR EACH ROW
EXECUTE FUNCTION pruefe_anfrage();
```

## Erläuterung

Der Trigger lässt eine Anfrage erst zu, wenn der Nutzer das Exemplar nicht selbst besitzt. Ein CHECK reicht dafür nicht. Er sieht nur die Zeile, die gerade eingefügt oder geändert wird. Die nutzer_id steht sowohl in exemplar als auch in anfrage. Ohne UPDATE ließe sich eine bestehende Anfrage nachträglich auf ein eigenes Exemplar umbiegen.

**Negativer Testfall** T17 · INSERT · erwartet P0001

```
1  INSERT INTO anfrage (status, nutzer_id,exemplar_id,art_id)
2  VALUES
3  ('Gestellt',4,10,1);
```

Data Output | Messages | Notifications

```
ERROR:  Nutzer 4 besitzt dieses Exemplar.
CONTEXT:  PL/pgSQL function pruefe_anfrage() line 6 at RAISE

SQL state: P0001
```

**Negativer Testfall** T18 · UPDATE · erwartet P0001

```
1  UPDATE anfrage SET exemplar_id=10 WHERE anfrage_id=7
```

Data Output | Messages | Notifications

```
ERROR:  Nutzer 4 besitzt dieses Exemplar.
CONTEXT:  PL/pgSQL function pruefe_anfrage() line 6 at RAISE

SQL state: P0001
```

# Indizes

Zwei partielle UNIQUE-Indizes

## Definition

```
CREATE UNIQUE INDEX ausleihe_aktiv
    ON ausleihe (exemplar_id)
    WHERE enddatum IS NULL;
```

```
CREATE UNIQUE INDEX anfrage_angenommen
    ON anfrage (exemplar_id)
    WHERE status='Angenommen';
```

## Erläuterung

Die Regeln betreffen nur eine Tabelle, brauchen jedoch einen Blick auf andere Zeilen. Ein CHECK sieht nur die eingefügte Zeile. Ein UNIQUE-Index prüft alle Zeilen.
ausleihe_aktiv verhindert, dass zwei aktive Ausleihen auf demselben Exemplar existieren. anfrage_angenommen verhindert, dass zwei angenommene Anfragen auf dasselbe Exemplar existieren.
Beide Indizes sind partiell. ausleihe_aktiv gilt für Ausleihen ohne Enddatum, anfrage_angenommen für angenommene Anfragen.

**Negativer Testfall** T19 · ausleihe_aktiv · erwartet 23505

```
1   UPDATE ausleihe SET enddatum = NULL WHERE ausleih_id = 1;
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "ausleihe_aktiv"
Key (exemplar_id)=(9) already exists.

SQL state: 23505
Detail: Key (exemplar_id)=(9) already exists.
```

**Negativer Testfall** T20 · anfrage_angenommen · erwartet 23505

```
1   UPDATE anfrage SET status='Angenommen' WHERE anfrage_id=8
```

Data Output | Messages | Notifications

```
ERROR:  duplicate key value violates unique constraint "anfrage_angenommen"
Key (exemplar_id)=(13) already exists.

SQL state: 23505
Detail: Key (exemplar_id)=(13) already exists.
```

# Datenbestand

13 Tabellen · 146 Datensätze

## Abfrage

```
SELECT 'ort' AS tabelle, COUNT(*) AS eintraege FROM ort
UNION ALL SELECT 'nutzer', COUNT(*) FROM nutzer
UNION ALL SELECT 'sprache', COUNT(*) FROM sprache
UNION ALL SELECT 'genre', COUNT(*) FROM genre
UNION ALL SELECT 'autor', COUNT(*) FROM autor
UNION ALL SELECT 'buch', COUNT(*) FROM buch
UNION ALL SELECT 'buch_genre', COUNT(*) FROM buch_genre
UNION ALL SELECT 'buch_autor', COUNT(*) FROM buch_autor
UNION ALL SELECT 'uebergabeart',COUNT(*) FROM uebergabeart
UNION ALL SELECT 'exemplar',     COUNT(*) FROM exemplar
UNION ALL SELECT 'anfrage',      COUNT(*) FROM anfrage
UNION ALL SELECT 'ausleihe',     COUNT(*) FROM ausleihe
UNION ALL SELECT 'bewertung',    COUNT(*) FROM bewertung
ORDER BY tabelle;
```

| | tabelle text | eintraege bigint |
|---|---|---|
| 1 | anfrage | 11 |
| 2 | ausleihe | 13 |
| 3 | autor | 12 |
| 4 | bewertung | 10 |
| 5 | buch | 12 |
| 6 | buch_autor | 13 |
| 7 | buch_genre | 18 |
| 8 | exemplar | 10 |
| 9 | genre | 13 |
| 10 | nutzer | 10 |
| 11 | ort | 10 |
| 12 | sprache | 12 |
| 13 | uebergabeart | 2 |

## Erläuterung

Alle Tabellen enthalten mindestens zehn Einträge. Ausnahme ist die Tabelle uebergabeart, da nur „Abholung“ und „Versand“ sinnvolle Einträge sind.

# JOIN: Aktive Ausleihen pro Nutzer

A1 · 2 Tabellen

## SQL-Abfrage

```
SELECT n.nachname,n.vorname,
COUNT(a.ausleih_id) AS aktive_ausleihen
FROM nutzer n
LEFT JOIN ausleihe a ON
n.nutzer_id=a.nutzer_id AND a.enddatum IS NULL
GROUP BY n.nutzer_id, n.nachname, n.vorname
ORDER BY aktive_ausleihen DESC, n.nachname;
```

## Erläuterung

Diese Abfrage zeigt jeden Nutzer mit der Anzahl seiner laufenden Ausleihen.
LEFT JOIN, damit auch Nutzer ohne Ausleihe erscheinen. Die Bedingung auf enddatum steht deshalb im ON und nicht im WHERE, da sie im WHERE nur genau diese Nutzer herausfiltern würde.
COUNT zählt ausleih_id anstatt *, weil * die Zeilen mit NULL-Werten mitzählen würde und damit jeder Nutzer eine 1 hätte, auch die ohne aktive Ausleihen.
GROUP BY ist nötig, weil neben dem COUNT noch Nachname und Vorname in der SELECT-Liste stehen.
ORDER BY, damit Nutzer mit mehr Ausleihen oben stehen.

## Ergebnis

| | nachname character varying (50) | vorname character varying (50) | aktive_ausleihen bigint |
|---|---|---|---|
| 1 | Dubel | Hugen | 1 |
| 2 | Kästner | Erich | 1 |
| 3 | Mustermann | Max | 1 |
| 4 | Goethe | Johann Wolfgang | 0 |
| 5 | Jones | Olivia | 0 |
| 6 | Kretzschmar | Ingo | 0 |
| 7 | Lenz | Siegfried | 0 |
| 8 | Lustig | Peter | 0 |
| 9 | Meier | Lisa | 0 |
| 10 | Schiller | Friedrich | 0 |

# JOIN: Überfällige Ausleihen

A2 · 4 Tabellen

## SQL-Abfrage

```
SELECT b.titel, n.nachname, n.vorname,
a.startdatum, a.ausleihdauer,
a.startdatum + a.ausleihdauer AS faellig_am
FROM nutzer n
JOIN ausleihe a ON n.nutzer_id=a.nutzer_id
JOIN exemplar e ON a.exemplar_id=e.exemplar_id
JOIN buch b ON e.isbn=b.isbn
WHERE a.enddatum IS NULL AND a.startdatum
+a.ausleihdauer<CURRENT_DATE
ORDER BY faellig_am;
```

## Erläuterung

Diese Abfrage zeigt alle laufenden Ausleihen, bei denen die vereinbarte Dauer abgelaufen ist, das Buch aber noch nicht zurückgegeben wurde.
Vier Tabellen wurden genutzt: buch, nutzer, ausleihe und exemplar. Diese werden benötigt, weil neben der Ausleihe auch der Name des Ausleihers und der Titel des Buches angezeigt werden sollen.
Die erste WHERE-Bedingung beschränkt auf laufende Ausleihen, die zweite auf solche, deren Fälligkeitsdatum vor dem heutigen Tag liegt.
Sortiert wird nach Fälligkeit, damit die am längsten überfällige Ausleihe oben steht.

## Ergebnis

| | titel<br>character varying (200) | nachname<br>character varying (50) | vorname<br>character varying (50) | startdatum<br>date | ausleihdauer<br>integer | faellig_am<br>date |
|---|---|---|---|---|---|---|
| 1 | Way of Kings | Mustermann | Max | 2026-09-20 | 21 | 2026-10-11 |
| 2 | Die Säulen der Erde | Kästner | Erich | 2026-09-28 | 14 | 2026-10-12 |

# JOIN: Durchschnittliche Bewertung je Verleiher

A3 · 4 Tabellen

## SQL-Abfrage

```
SELECT n.nachname, n.vorname,
ROUND(AVG(b.sterne),2) AS sterne_durchschnitt,
COUNT(b.bewertung_id) AS anzahl_bewertungen
FROM bewertung b
JOIN ausleihe a ON b.ausleih_id=a.ausleih_id
JOIN exemplar e ON a.exemplar_id=e.exemplar_id
JOIN nutzer n ON e.nutzer_id=n.nutzer_id
GROUP BY n.nutzer_id
ORDER BY sterne_durchschnitt DESC ,
anzahl_bewertungen DESC, n.nachname;
```

## Erläuterung

Diese Abfrage zeigt für jeden Nutzer, der Bücher verleiht, den Durchschnitt der Sterne und die Anzahl der Bewertungen.
Vier Tabellen werden genutzt: bewertung, ausleihe, exemplar und nutzer. Diese sind nötig, weil die bewertung an ausleihe hängt, die ausleihe an exemplar, das exemplar wiederum an dem nutzer.
Wichtig ist der letzte JOIN, ausleihe und exemplar haben beide die Spalte nutzer_id, aber es sind unterschiedliche Personen. In ausleihe der Ausleiher und in exemplar der Besitzer. Bewertet wird der Besitzer, daher der Join über e.nutzer_id.
Die Anzahl wird mit ausgegeben, weil ein Durchschnitt aus einer einzelnen Bewertung wenig aussagt. Sortiert wird nach dem höchsten Sternendurchschnitt. Bei gleichem Wert entscheidet die Anzahl der Bewertungen, danach folgt der Nachname.

## Ergebnis

| | nachname<br>character varying (50) | vorname<br>character varying (50) | sterne_durchschnitt<br>numeric | anzahl_bewertungen<br>bigint |
|---|---|---|---|---|
| 1 | Kretzschmar | Ingo | 4.50 | 2 |
| 2 | Meier | Lisa | 4.50 | 2 |
| 3 | Goethe | Johann Wolfgang | 4.00 | 1 |
| 4 | Jones | Olivia | 3.67 | 3 |
| 5 | Dubel | Hugen | 3.00 | 2 |

# JOIN: Verfügbare Bücher

A4 · 6 Tabellen

## SQL-Abfrage

```
SELECT b.titel, n.nachname, n.vorname,
o.postleitzahl, o.stadt,
STRING_AGG(g.genre, ', ' ORDER BY g.genre) AS genres
FROM exemplar e
JOIN buch b ON e.isbn=b.isbn
JOIN buch_genre bg ON b.isbn=bg.isbn
JOIN genre g ON bg.genre_id=g.genre_id
JOIN nutzer n ON e.nutzer_id=n.nutzer_id
JOIN ort o ON n.postleitzahl=o.postleitzahl
WHERE e.status='Verfuegbar'
GROUP BY b.isbn, n.nutzer_id, o.postleitzahl,
e.exemplar_id
ORDER BY o.postleitzahl, b.titel;
```

## Erläuterung

Diese Abfrage zeigt alle verfügbaren Exemplare mit Titel, Genre, Besitzer und Standort, sortiert nach Postleitzahl und Titel.
Sechs Tabellen werden genutzt: exemplar, buch, buch_genre, genre, nutzer und ort.
Diese werden benötigt, weil exemplar in zwei Richtungen zeigt: zu buch, buch_genre und genre für das Buch selbst, sowie zu nutzer und ort, um zu zeigen, wo das Exemplar liegt.
STRING_AGG fasst die Genres zusammen, damit Bücher mit mehreren Genres nicht mehrfach aufgelistet werden. Die Postleitzahl gruppiert nach Ort, berechnet aber keine Entfernungen.

## Ergebnis

| | titel character varying (200) | nachname character varying (50) | vorname character varying (50) | postleitzahl character (5) | stadt character varying (100) | genres text |
|---|---|---|---|---|---|---|
| 1 | Krieger des Feuers | Jones | Olivia | 22767 | Hamburg | Fantasy |
| 2 | Die Lügen des Locke Lam… | Dubel | Hugen | 80636 | Muenchen | Fantasy, Krimi |
| 3 | Way of Kings | Meier | Lisa | 80636 | Muenchen | Fantasy |
| 4 | Medicus | Goethe | Johann Wolfgang | 99423 | Weimar | Historisch, Rom… |

# Zusammenfassung

Für die Datenbank der Buchtausch-App wurde zunächst ein ERM-Diagramm entworfen, um die Beziehungen zwischen den Entitäten besser darzustellen. Die Umsetzung folgte dann den Fremdschlüsselabhängigkeiten. Tabellen ohne Fremdschlüssel wurden zuerst angelegt, danach die Tabellen, die auf sie verweisen. Die genaue Reihenfolge entspricht der Anordnung der Entitäten in der Präsentation.
Insgesamt entstanden 13 Tabellen und ein Trigger auf bewertung, zwei Trigger auf ausleihe und einer auf anfrage. Jede Tabelle enthält mindestens 10 Datensätze, mit Ausnahme von uebergabeart: Dort sind nur zwei Werte sinnvoll, nämlich „Versand“ und „Abholung“.
Die Trennung von Buch und Exemplar wurde bewusst vorgenommen, da ein Werk in mehreren physischen Objekten vorliegen kann. Eine Tabelle, die beides zusammenfasst, würde Redundanz erzeugen. Da relationale Datenbanken eine N:M-Beziehung nicht direkt abbilden können, braucht es dafür zwingend eine dritte Tabelle. Für die Beziehungen zwischen Buch und Autor sowie zwischen Buch und Genre wurde daher jeweils eine Zwischentabelle angelegt.
In den meisten Tabellen genügte ein CHECK. In vier Fällen braucht es einen Trigger, und zwar aus zwei Gründen. Bei bewertung, ausleihe und anfrage reicht die eingefügte Zeile zur Prüfung nicht aus, weil der entscheidende Wert in einer anderen Tabelle steht. Im vierten Fall wird dort nicht geprüft, sondern geschrieben: Der Status des Exemplars folgt automatisch dem Zustand der Ausleihe. Zwei Regeln betreffen nur eine Tabelle, brauchen jedoch einen Blick auf andere Zeilen. Dies kann ein CHECK nicht. Daher wurden zwei UNIQUE-Indizes benötigt: einer für ausleihe und einer für anfrage. Diese stellen sicher, dass höchstens eine aktive Ausleihe je Exemplar und höchstens eine angenommene Anfrage je Exemplar existieren.
Eine Schwierigkeit lag in der Reihenfolge innerhalb des Installationsskripts: Der Status der Exemplare lässt sich erst nach den Ausleihen setzen; dies wird durch den trg_status gelöst.
Die vier Abfragen zeigen, dass sich aus den verknüpften Tabellen konkrete Fragen der App beantworten lassen.